CST2110 Task 1 Reflective Report

Three-player dice game

Beejalben Amitkumar Patel
M01035595

This report is about how the game was planned, built, changed and tested. It is not a description of the screens. The examples below are taken from the program: the column 8 case in the brief, a tied column, and full games that could be repeated because the dice used a fixed seed.

Planning

I wrote the rules as checks before I chose any classes. There are three players and eleven rounds. A turn is two dice, then one column from 2 to 12. A player cannot put a second total in a column they have already used. In round 1 the three players must pick different columns. From round 2 a column may be shared. When the third total is placed, the highest total wins that column's number as points. If two or more players share the highest total, the column scores nothing. A total beaten by a higher one is shown as X. At the end, the highest score wins. A tie is stated. There is no play-off. The program also lists every column.

Writing it that way stopped two mistakes. The Scores column is not the sum of the dice. The points are the headers 2 to 12. A column can also be completed in the middle of a round, as soon as the third player chooses it, so scoring cannot wait until the end of round 11. I split the work into the throw, the sheet, and the keyboard. The wording can change without the scoring changing with it.

How the program was built

The submission is a NetBeans Ant project. In nbproject/project.properties both javac.source and javac.target are 11. The main class is dicegame.DiceGame. I kept to Java 11. There are no records and no text blocks, so a lab machine on JDK 11 can compile the same source.

Dice and DiceThrow produce the two faces and the total. Player stores the name and the points won, and nothing else. ScoreCell stores one dice total and a flag for whether that total has been crossed out. ScoreSheet holds a ScoreCell array of 3 by 11. Column 2 is stored at index 0, so column c is index c - 2. I used a fixed array rather than a list because this table cannot grow. ColumnResult stores the totals, who led, and whether points were given, so the message at the time and the closing summary cannot disagree.

Game is the only class that reads the keyboard. It waits for Enter, rolls, and then stays in a loop until the column is legal. The sheet is not given a value until those checks have passed. Each bad entry has its own message: empty input, text, a number outside 2 to 12, a column this player has already used, and a column someone else took in round 1.

The scoring method is the part that had to be exact. It finds the best total. If one player has it, that player gains the column number, not the dice total. Every total below the best is struck. If two or three players share the best total, nobody gains points, but a strictly lower total is still struck. The brief says both that a tie does not score and that an entry defeated by a better result is crossed out. Those are not the same rule. Round 1 is not stored on the sheet. Game keeps a boolean array for that round only. I did not want the sheet to carry a rule that lasts for one round.

What changed after the first version

My first reading of a tie was to cross nothing out. The sentence about a better result made me change

it. A column of 9, 9 and 4 now leaves both 9s on the grid, shows X for the 4, and adds no points. I ran that case on its own before trusting it.

I also replayed the brief's column 8 example directly on the sheet, not by hoping the dice would match. Player 1 places 10, Player 2 places 10, Player 3 places 11. Player 3's score becomes 8. The two 10s become X. The earlier cells stay: 5 in column 2, 7 in column 5, and 8 in column 6. That is Figure 5. The column was not scored when only two players had used it. It scored on the placement that filled it.

The first table was only numbers and was hard to scan. I added a bottom border, a line under the grid explaining that X is a beaten total and that Scores are column points, and a text drawing of the two dice. The drawing uses plus and minus signs, so it is still a console program. The round underline was a fixed row of dashes, which was shorter than "Round 10 of 11". It now follows the length of the heading. On the last turn the program says which single column is left, after a list of one column proved easy to miss.

Testing

Ant compile was run on JDK 11. javap on DiceGame.class reports major version 55, which is the Java 11 class format. The project was cleaned afterwards so the submission does not contain class files.

A script then played the console and kept a second copy of the rules. After every placement it read the printed table and compared each cell, each X and each score. Seeds 1 to 40 all matched. Those runs contained 72 tied columns and one tied game. Seed 2, column 2, was a tie on 6 between Player 1 and Player 3. Player 2 had 3, which was crossed out, and nobody scored. Seed 34 finished 31, 15 and 31, and the result line names Player 1 and Player 3. Seed 1 had a single winner in every column and finished 22, 28 and 27. Those add to 77, and 2 + 3 + ... + 12 is also 77, so the points really were the column headers.

On seed 1 the script also sent a blank line, the word nope, 1, 13 and 2.5. In round 1 it tried to take the column Player 1 had just used. In round 2 it tried to reuse column 2. Each attempt was rejected and the game continued. The same seed played twice produced the same text. Closing the input after the first Enter printed "Input has ended, so the game has stopped." and did not throw an exception. I did not treat a single happy-path game as enough, because a tie and a bad column are where this program is most likely to be wrong.

Reflection

The split between Game and ScoreSheet is the decision I would keep. A bad column never reaches the array, so a mistake in the prompt cannot write over a cell. Putting c - 2 in one method also matters. That off-by-two error would score the wrong column and it would be hard to see in the grid.

The weak spot is how much is fixed. The player count and the round count are constants, and the sentence for a tied game is written for two or three names. A fourth player would need more than a new constant. ScoreSheet also adds the points onto Player itself. That keeps the points next to the comparison, and it means a column cannot be marked finished and then forget the score. It does mean the sheet changes another object. Returning the result and letting Game add the points would be a cleaner split. I left the update in the sheet on purpose, and I would revisit it if the scoring rule grew.

There is no confirm step. Once the column is entered, it is stored. The brief does not ask for undo, and an undo would have to undo a column that might already have been scored. At a shared keyboard a mistype is still the likely mistake. The next change I would make is a yes-or-no check before the total is locked.

X matches the brief, and the cell still holds the number so the summary can quote it. During the game

that number is hidden. I would not print both X and the number in the same cell. The columns would stop lining up. The summary is the right place for the hidden totals, which is where they are now.

The rules were tested more carefully than the look of the NetBeans window on a lab PC. The layout uses spaces and ASCII lines so it does not depend on a special font, but I have not watched it in the lab colour scheme. That is the check I would do on the machine that will mark it, after Clean, before the zip is uploaded. The names stay as Player 1, Player 2 and Player 3 because that is the table in the brief. Asking for names would have been extra behaviour the marker did not ask to see.

Against the task criteria

The project is Java 11, configured as a NetBeans Ant build, and cleaned so the class files are gone. It builds with Ant. It plays eleven rounds, enforces the round 1 rule, scores a column when it is filled, handles ties, crosses out beaten totals, keeps a running score, and ends with a winner or a named tie plus a column summary. The console table is aligned and the dice are drawn in text. The classes listed above are the modular split, and the fixed array plus the single pass over a column is the structure behind the scoring. The input loop is what the script threw bad data at, and those runs did not raise an exception. The comments that carry the design are the scoring rule in ScoreSheet and the reason round 1 lives on the game rather than the sheet. The sections above are the plan, the build, the change to the tie rule, the tests, the reasons for those choices, and the limits I would change next.